



SR-LLM: An incremental symbolic regression framework driven by LLM-based retrieval-augmented generation

Zelin Guo^{a,1} , Siqi Wang^{a,1} , Yonglin Tian^b , Jing Yang^b , Hui Yu^c , Xiaoxiang Na^d, Levente Kovács^e , Li Li^{a,2} , Petros A. Ioannou^{f,2}, and Fei-Yue Wang^{g,h,2}

Affiliations are included on p. 11.

Edited by Jeffrey Ullman, Stanford University, Stanford, CA; received June 27, 2025; accepted November 10, 2025

Symbolic regression (SR) has regained research prominence as deep learning advancements accelerate the search for analytical models from observational data. However, the vast search space often hinders existing algorithms to yield complex analytical expressions. We present SR-LLM, an SR framework integrating retrieval-augmented generation mechanisms based on large language models (LLM) to achieve incremental learning. Specifically, our framework is capable of leveraging accumulated prior knowledge and past exploration results from external knowledge bases to retrieve the most relevant information for current regression tasks. It first composes prior information into small symbolic groups with the assistance of the LLMs and then utilizes deep reinforcement learning to combine these groups to formulate complex yet explainable analytic expressions that are more easily understood by humans. The capability for efficient knowledge utilization enables our framework to integrate all previous human experiences and exploration results, effectively learning by standing on the shoulders of giants. To validate the effectiveness of our proposed method, we not only test the framework on popular symbolic regression benchmarks but also extend its application to a domain where the explicit optimal model remains controversial: how to analytically describe human car-following behavior based on observed vehicle trajectories? Experiments confirm that our method outperforms on standard benchmarks, successfully rediscovers famous traditional car-following models and discovers new models from empirical trajectory data, achieving both fitting effectiveness and interpretability.

symbolic regression | retrieval-augmented generation | large language models

Symbolic models have been instrumental in scientific progress for millennia (1), offering concise representations, clear interpretations, and strong generalizability (2). However, automatically distilling noisy data into knowledge in the form of analytical laws remains challenging across many scientific and engineering fields. In the 1990s, some researchers criticized that artificial neural networks were unable to discover fundamental physical laws such as the law of universal gravitation, while studies in the last ten years suggest symbolic regression (SR) as a potential solution (3–5). SR aims to simultaneously find the structure and determine the parameters of an analytical expression that best describes observed data in an interpretable manner. Unlike traditional regression, which fits parameters to a predefined form, SR flexibly uncovers the models that human experts might not envision.

However, several significant challenges must be addressed before SR can be widely used. The first obstacle is the infinite number of possible analytical expressions that should be searched. We need to identify the analytical relations within the observed data and focus on meaningful expressions. Moreover, we need to develop principles for identifying such relationships, as we cannot create them out of thin air. While SR can rediscover simple formulations like the universal gravitation rule, few studies have successfully applied it to complex, unconventional models. Some SR methods applied tree search algorithm to attack this problem (6), but the search space remains prohibitively large due to the lack of useful regression knowledge gained during the process.

The second obstacle lies in constructing complex yet interpretable analytical expressions. Many existing SR methods prioritize discovering parsimonious physical models from observed data. As a result, some complex analytical expressions are taken as trivial or meaningless and are thus omitted. However, in many applications, delicate expressions are necessary to accurately describe perplexing data patterns. We also aim to keep these delicate models explainable and often decomposable. How to establish complex yet meaningful analytical expressions remains an open challenge.

Significance

Scientists have long sought to derive models from extensive observational input–output data, ensuring these models accurately capture the underlying mapping from inputs to outputs while remaining interpretable to humans through clear meanings. Also, this has long been the ultimate goal of symbolic regression. The primary contribution of our work lies in leveraging the extensive knowledge base and reasoning capabilities of large language models to enhance symbolic regression, thereby obtaining analytical models that are both accurate and highly interpretable. Extensive experiment results indicate that our method consistently outperforms existing approaches on standard benchmarks while yielding more interpretable models, thus confirming the significant potential of large language models in improving both the fitting performance and interpretability of symbolic regression.

The authors declare no competing interest.

This article is a PNAS Direct Submission.

Copyright © 2025 the Author(s). Published by PNAS. This open access article is distributed under [Creative Commons Attribution-NonCommercial-NoDerivatives License 4.0 \(CC BY-NC-ND\)](#).

¹Z.G. and S.W. contributed equally to this work.

²To whom correspondence may be addressed. Email: li-li@tsinghua.edu.cn, ioannou@usc.edu, or feiyue.wang@ia.ac.cn.

This article contains supporting information online at <https://www.pnas.org/lookup/suppl/doi:10.1073/pnas.2516995122/-DCSupplemental>.

Published December 22, 2025.

The third obstacle is the time-consuming nature of parameter calibration, particularly when dealing with noisy data. The influence of noise in data on regression model structure should also be carefully handled. As the analytical expressions grow more complex, parameter calibration becomes more time-consuming, slowing the search in analytical expression space. Moreover, calibrating complex symbolic regression models often involves nonlinear optimization problems, making it difficult to find global minimum.

In this paper, we introduce the SR framework to integrate retrieval-augmented generation (7) (RAG) for incremental learning on both synthetic and real-world data. RAG, a proven technique in large language models (LLMs), has demonstrated significant success in various fields (8–11). Due to their extensive parameter scales and rich pretraining corpus, LLMs exhibit unparalleled semantic understanding and context-based reasoning capabilities. On this basis, RAG can further enhance the answer quality of LLMs in specific domains. By retrieving knowledge from external knowledge bases, RAG embeds external knowledge into prompts, enriching the knowledge base of large language models and improving output reliability in specific fields.

Although we have also noted some other efforts applying large language models to symbolic regression, many of these approaches lack sustained knowledge accumulation (12–15), relying solely on data and models within a single run. This limits their ability to effectively accumulate and utilize past knowledge and experience, restricting long-term model evolution and optimization of the models. While there are some efforts that maintain an externally accumulated long-term knowledge base (16), they do not efficiently prune or simplify the search space, making searches relatively inefficient.

Inspired by the remarkable commonsense reasoning, inference capabilities, and generative enhancement mechanisms of LLMs and RAG, we aim to design a SR framework that fully leverages accumulated knowledge from human history and prior exploration to discover high-quality symbolic models, akin to AlphaGo (17), which defeated the world Go champion in 2017. Building on this vision, we propose a novel symbolic regression framework, named SR-LLM, for LLMs and RAG to symbolic regression. SR-LLM employs deep reinforcement learning (DRL) as the foundation for search and incorporates a novel two-stage calibration algorithm to flexibly balance accuracy and efficiency in parameter estimation, while integrating LLMs and RAG to enhance both the fitting performance and interpretability of the search process. By maintaining an updatable knowledge module, our framework not only leverages the expert knowledge initially embedded within the module but also capitalizes on past experiences during model exploration, facilitated by the LLM, and feeds successful search outcomes back into the module. We utilize LLMs to infer new composite symbols, effectively reducing the number of nodes in the search tree and significantly narrowing the search space. Notably, the LLMs provide strong interpretability guarantees for these newly generated symbol combinations, which is manifested not only in SR-LLM's ability to output intermediate reasoning processes and results in natural language during the search, but also in its capacity to help human experts understand the logical formation of newly combined symbols and discovered models. In this aspect, SR-LLM surpasses all existing symbolic regression approaches, which merely present the formal structure of symbol expressions. In contrast, SR-LLM explains why symbols should be combined in specific ways, thereby rendering analytical expressions substantially more comprehensible to humans (18, 19).

Our experiment is conducted in three steps: first, we test our framework on 100 expressions of varying complexity without physical dimensions, named “Fundamental-Benchmark,” to demonstrate its fundamental search capacity without the LLM. Second, we test our method on the “Feynman-Benchmark,” which includes 100 classical physics formulas with clear physical meanings, to evaluate SR-LLM's ability to leverage existing human knowledge and past explorations for improving search performance. Finally, we test our framework's performance on modeling longitudinal car-following behaviors of human drivers using the real-world Next Generation SIMulation (NGSIM) dataset. The reason for choosing models in this domain for symbolic regression is that car-following trajectory data not only contains significant measurement noise (20, 21) but also requires consideration of numerous different factors (22, 23). This increases model complexity, making it challenging to regress new models that balance performance, interpretability, and simplicity. Test results show our approach excels in basic search capabilities, utilizes external knowledge to enhance search efficiency, and finds universal and interpretable car-following models from empirical data, demonstrating its great potential when applied to symbolic regression across various disciplines. Based on our experimental results, we believe that an LLM incorporating all existing models designed by human experts over the past millennia could further enhance symbolic regression and potentially revolutionize machine-assisted scientific discovery in other domains.

Results

The Framework of SR-LLM. Representing complex symbolic systems with tree structures is an ancient yet vibrant idea. Symbolic trees are not merely forms of expression, but also carriers of reasoning mechanisms. This view reflected the prevailing perspective in the field of symbolic AI at the time, establishing symbolic trees as the core representational method in some early AI systems (24–26). In recent years, with the rise of artificial intelligence, symbolic trees have begun to be applied in various complex intelligent systems (5, 27), owing to their effectiveness in integrating symbolic reasoning with the representational power of neural networks, thereby addressing the limitations of traditional artificial intelligence in terms of interpretability and reasoning capabilities. In our proposed SR framework, we embrace this conceptual heritage by representing the sampling tree as a binary tree data structure. Each node represents a symbol in a continuously updating expression library, i.e., a variable, a constant, an operation (e.g., $+$, $\times$, $\sin$), or a combination (a group of variables, constants, and operations; regarded as a unified entity such as $v - v_0$); see Fig. 1A for an illustration. Operators form nonleaf nodes, with the number of branches matching the number of operands, while other symbols form leaf nodes. Generating a new candidate analytical model involves adding new symbols to the sampling tree to formulate a new tree.

Introducing a binary tree model offers two primary advantages. First, using the standard tree preorder traversal algorithm, we can easily convert the sampling tree of the candidate model into a fixed-size vector representation. As shown in Fig. 1A, the vector is initially populated with sampled symbols, and the remaining elements are filled with a “NULL” placeholder. We can therefore apply an appropriate sequence prediction algorithm to predict the next symbol added to the sampling tree based on its vector representation. The updated sampling tree can then serve as input for further predictions, creating a self-regression problem similar

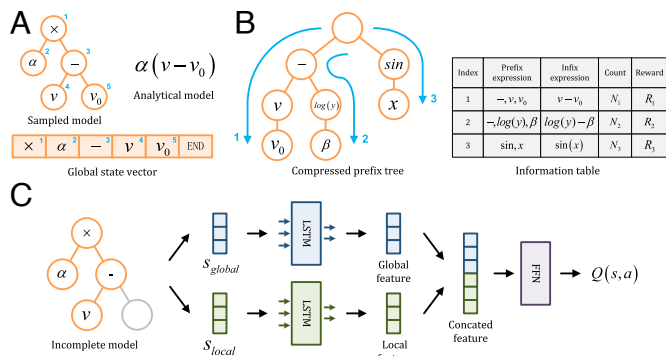


Fig. 1. An illustration of sampling tree and sampled tree for candidate model search. (A) A sampling tree and its corresponding vector presentation of the candidate analytical models. (B) A sampled tree and the associated candidate models that had been sampled. (C) The structure of the policy network used for model search.

to those in applications like natural language text generation [e.g., ChatGPT (8)]. Second, the binary tree structure simplifies the retrieval of structural information, such as parent and sibling nodes, enriching the input data for symbol prediction.

We also design a radix tree (compressed prefix tree) data structure (28) to store all sampled candidate models, aiming to leverage historical sampling information to enhance the search process in a principled and efficient manner. Each node in the radix tree represents a symbol from the expression library, and each root-to-leaf path corresponds to a candidate model. This design offers three key advantages. First, we can efficiently store several related candidate models and their corresponding sampling prefixes in one radix tree. Fig. 1B gives an example where the radix tree contains three candidate analytical models, which can be retrieved using the standard preorder traversal

algorithm. Inspired by the Monte Carlo Tree Search (MCTS) algorithm (29), we conveniently record the visitation frequency of all sampling prefixes within the radix tree framework and increase the exploration weight of rarely visited symbols (SI Appendix, Eq. 5). This enhances the global search capability and helps prevent premature convergence. Second, the radix tree facilitates incremental learning in building sampling trees. Symbols in excellent candidate models can be easily identified in prefix trees. By continuously discovering and adding more complex and semantically rich combinations, we can gradually build highly complex analytical expressions in a step-by-step manner. Third, the calibration and evaluation time are significantly reduced, as calibrated parameter values and evaluated scores of models are stored and reused. Consequently, identical sampled models do not require parameter recalibration or score recomputation, which saves time and computational costs.

The core of our SR framework can be divided into four parts, as illustrated in Fig. 2. The framework sequentially executes these four parts to sample new analytical models and meanwhile gain more sampling knowledge.

In “Sampling-Part,” we employ deep reinforcement learning (DRL) to learn and choose whether and where to add the candidate symbol into an existing sampling tree. Specifically, DRL uses a deep policy network that outputs the probability value of selecting a particular symbol. We use DRL for two main reasons: 1) Each time one symbol is added to the sampling tree, DRL continuously refines its sampling strategy based on rewards from environmental interactions. 2) DRL aligns well with the LLM-based knowledge-driven search paradigm. The LLM uses incremental learning to construct beneficial combinations from search results, reducing DRL search complexity and enhancing analytical model interpretability. In the rest of this paper, the parameters of the deep learning models in policy network

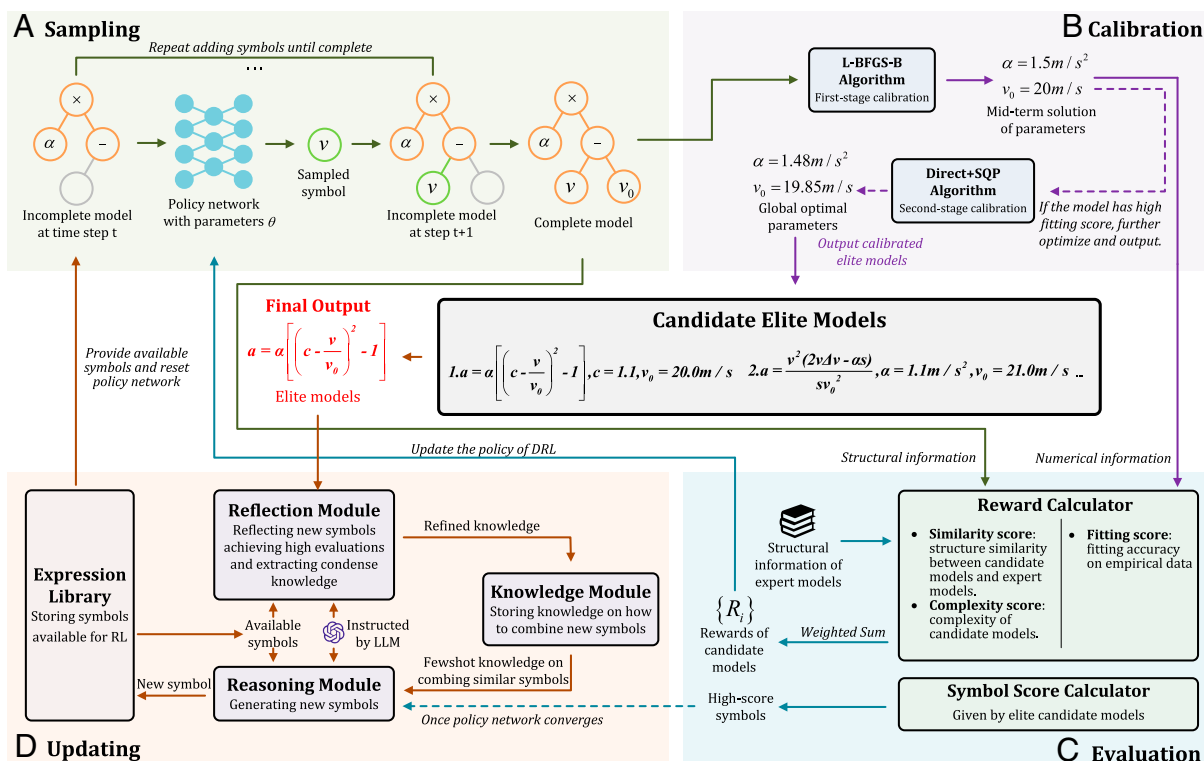


Fig. 2. The structure of SR-LLM.

are denoted as $\theta \in R^P$, where P represents the parameters dimension.

The state s_t of the DRL comprises two components, which is also serves as the input to the policy network: a global state vector s_t^{global} and a local state vector s_t^{local} , both associated with the search tree.

1) The global state vector s_t^{global} describes the whole search tree sampled at step t . Using the standard preorder traversal algorithm, the search tree can be directly converted into vector s_t^{global} , as illustrated in Fig. 1A. Each nonempty element of vector s_t^{global} denotes a sampled symbol. The first empty element is assigned as “END,” and the rest empty elements are uniformly assigned as “NULL.” If a node represents a combination, it may occupy multiple elements in the vector s_t^{global} .

2) Following the approach of previous symbolic regression tool PhySO (5), we use the local state vector s_t^{local} to store the local sampling information. The local state vector aggregates information from currently sampled expressions, including sequence numbers of the parent node, sibling nodes, and the previously sampled symbol, as well as the current placeholder state, required physical unit, and physical units of the parent node, sibling nodes, and previously sampled symbol. All values are numerically encoded.

The global state vector enables the policy network to capture holistic patterns across the search tree, while the local state vector helps preserve fine-grained structural features of high-performing expressions discovered thus far, the exploration of promising compositional structures and particularly enhancing sensitivity to physical unit constraints. Notably, based on the distinct characteristics of global and local state vectors, we can flexibly select input components for subsequent different symbolic regression tasks (SI Appendix, Table S2). For example, in the car-following experiment, which leverages both global and local state representations, SR-LLM achieves a richer and more informative perception of the state space. Additional experimental results are provided in SI Appendix, Table S7 and Fig. S14.

The allowable action $a_t \in A$ of deep reinforcement learning involves selecting a symbol from the expression library and adding it to the search tree, where A denotes the action space. To determine the optimal selection, our framework uses two long-short term memory (30) (LSTM) networks to encode information from the global and local state vectors, respectively, as shown in Fig. 1C. When both state vectors are input, the outputs of the LSTM networks are concatenated into a feature vector $s_t^{feature}$. We choose LSTM networks because we aim to memorize the existing symbols in the search tree while calculating what the next symbol should be. Then, feature vector $s_t^{feature}$ will be fed into a feedforward network (FFN) to generate the value $Q(s_t, a_t | \theta)$ of taking action a_t in state s_t . A higher value $Q(s_t, a_t | \theta)$ indicates a greater probability of selecting action a_t . However, $Q(s_t, a_t | \theta)$ is not directly to update the search tree, since we need to consider some other a-prior constraints, e.g., guaranteeing the analytical model physically legal, and not allowing redundant expressions to appear. Details on applying these a-prior constraints are explained in *Materials and Methods*.

In each execution round, we generate multiple candidate analytical models via self-regression before completing the “Sampling-Part.” Once a feasible analytical model is found, we will record the associated sampling tree and also save it in a sampled tree. If the newly generated model has appeared in the sampled tree, we will only update its sampled count; otherwise,

we will create a new prefix path and proceed with subsequent calibration and evaluation. The “Sampling-Part” is ended if all candidate models in the batch have been sampled or reached their maximum sampling length.

In the “Calibration-Part,” we perform a two-stage calibration procedure to efficiently screen newly generated analytic expressions and reduce the fitting error of symbolic models. In the first stage, we first predefine feasible ranges for the parameters and initialize their values at the midpoint of these intervals. Subsequently, we employ the L-BFGS-B algorithm (31), an extension of the classical L-BFGS (32) method that incorporates bound constraints during optimization, within a fixed computational budget. We prefer L-BFGS-B over the standard L-BFGS primarily because, in symbolic regression, optimizing free constants often requires constraining parameters to physically plausible or domain-interpretable ranges. This design is particularly critical in applications with explicit meaning. However, many conventional symbolic regression approaches overlook this consideration, performing unconstrained optimization that may yield models with good fit but poorly interpretable or even physically implausible parameter values. Notably, L-BFGS-B naturally reduces to the standard L-BFGS when bound constraints are unnecessary, thereby offering greater generality and flexibility.

In the first calibration stage, an initial parameter estimate is rapidly obtained. If this preliminary solution results in a fitting error exceeding a predefined threshold for the candidate analytic model, we assume that further refinement is unnecessary and discard the candidate. This strategy prioritizes computational efficiency by allocating minimal resources to low-quality models, effectively filtering them out early. Otherwise, the candidate proceeds to the second calibration stage, where a hybrid “Direct+SQP” calibration method (33) is employed. This approach, designed for higher precision in optimization problems within a given range, aims to identify the globally optimal parameters within the prescribed bounds. Notably, the method supports not only local calibration for data points without mutual coupling but also global calibration for coupled data pairs, such as temporally correlated trajectory data in dynamical physical systems. This two-stage calibration design enables SR-LLM to quickly identify models that fit the observed data well, while substantially increasing the likelihood of finding globally optimal parameter values for high-quality candidates within a longer yet still acceptable computation time. Detailed experimental results are presented in SI Appendix, Table S3 and Fig. S1.

In “Evaluation-Part,” we score each newly generated analytical model using a linearly weighted sum of three criteria: r_{fit} the score value of model fitting errors obtained after the Calibration-Part, $r_{similarity}$ the similarity score between the generated analytical models and all the preselected models designed by human experts, and $r_{complexity}$ the complexity score of the models. All three score values are normalized into range $[0, 1]$, with values closer to 1 indicating better performance. We include similarity to expert models to encourage the framework to learn from human experts, which can be regarded as a form of expert knowledge. Complexity is considered to avoid overly complex formulas when fitting real-world empirical data. The final weighted score can be represented as $R = w_{fit}r_{fit} + w_{similarity}r_{similarity} + w_{complexity}r_{complexity}$. Notably, to address diverse symbolic regression tasks, our method flexibly balances the weights of the three score components (details in SI Appendix, Table S2). For example, in the car-following experiments, all three components are combined to guide SR-LLM in leveraging prior domain knowledge while discouraging

overly complex formulas, whereas in the benchmark experiments, the evaluation primarily relies on r_{fit} to exactly recovery.

After evaluation, we update the DRL policy network by R , which serves as reward. Initially trained by randomly selecting symbols from the expression library, the network is refined using policy-gradient reinforcement learning, employing two strategies consequently. The detailed algorithmic procedure is provided in [SI Appendix, Algorithm 1](#).

1) Risk-seeking policy gradient. Inspired by Petersen et al. (34), we reinforce only the top-performing models in each batch and their corresponding sampling paths in the radix tree. Poorer models are not penalized, as symbolic regression focuses solely on discovering the best-performing expression. However, while the risk-seeking policy gradient can quickly and straightforwardly reinforce the best samples, it often struggles to escape local optima once it has settled into one (35), limiting the search for a superior global solution.

2) Soft Actor-Critic (SAC). To address stagnation, we switch to the second strategy using SAC algorithm. SAC reuses past experience from a replay buffer and can more efficiently and robustly approximate the global optimum in DRL (36). During the risk-seeking phase, experience transitions are stored in the replay buffer to ensure sufficient samples for SAC; while concurrently optimizing the critic network and entropy regularization factor, preparing for a seamless transition to SAC. Here, the actor network in SAC is also the DRL policy network. When SAC takes over, we use prioritized experience replay (PER) to further improve sample efficiency (37): samples with larger temporal difference (TD) errors are given higher sampling probabilities. SAC iteratively refines the policy network, gradually approaching the globally optimal strategy.

The final weighted score R of all the sampled analytical models is recorded in the leaves of the sampled radix trees. If a sampled model already exists in the sampled radix trees, its sampling frequency is incremented. Otherwise, this model completes a path expansion from the root to a leaf node, and its score is recorded to reuse.

We also record all the elite candidate models discovered so far, as they may contain useful combinations for describing the observed data. The maximum number of elite models is predefined. New candidate models with higher scores replace older ones. After scoring expressions, we compute symbol scores as the weighted average of scores for all expressions containing the symbol, weighted by its contribution to each expression's score. High-score symbols are selected as input for the "Updating-Part."

In the Updating-Part, we leverage LLMs to generate novel symbolic expressions for updating the expression library, while simultaneously extracting knowledge about effective symbolic model construction from elite expressions sampled by DRL, constituting a core innovation of our work. The update process comprises three interconnected modules: knowledge, inference, and reflection, as illustrated in Fig. 3A, with detailed examples provided in Fig. 3 B–D.

The Knowledge module stores valuable insights regarding beneficial symbolic compositions. It functions as an external knowledge base that combines expert knowledge on symbolic combinations with high-performing patterns accumulated by SR-LLM during prior exploration phases. This module can be queried based on the semantic content of high-scoring symbols identified in the evaluation stage, enabling retrieval of the most relevant prior knowledge. As shown in Fig. 3B, each knowledge entry includes five different aspects: source (origin of the knowledge), key (semantic meaning of the source symbol

used for retrieval), target (semantic meaning of the resulting composite symbol), content (the knowledge itself), and reflection (assessment or meta-knowledge about the entry). These stored insights are subsequently utilized in the reasoning module to construct question-answer(Q&A) pairs that enhance the search process.

The Inference module operates on high-scoring symbols obtained from the evaluation-part and retrieves few-shot examples of semantically similar symbols from the knowledge module. It consists of a prompt generator and a new symbol extractor, which together guide the LLM in expanding the expression library. Specifically, when generating a new prompt, the system first queries the knowledge module to retrieve contextual information and prior successful instances related to the target symbol. This retrieved knowledge is then integrated into the prompt by the prompt extractor, enabling the LLM to better understand task requirements and creatively generate new, semantically meaningful symbols. The generated symbols will be extracted by the symbol extractor and update to the expression library. RAG plays a pivotal role here, effectively combining retrieval from an existing knowledge base with generative capabilities to produce innovative symbolic expressions. A concrete example of such a Q&A pair is shown in Fig. 3C, where SR-LLM generates a new symbolic construct through a chain-of-thought process, integrating information from few-shot conversations and information of high-score symbols(including the description of the symbol itself and elite models sampled from DRL containing it).

Once newly generated symbols are explored and validated by DRL, the Reflection module further engages the LLM to analyze why high-performing models succeed. This reflective analysis captures the underlying principles behind elite expressions discovered by DRL, deepens understanding of existing symbolic representations, and yields actionable insights for future symbolic optimization. The distilled knowledge is then incorporated back into the knowledge module, establishing a continuous learning loop. As depicted in Fig. 3D, by hierarchically decomposing elite expressions, SR-LLM extracts their internal compositional logic and formalizes this understanding into structured knowledge entries for long-term retention. This integrated update mechanism enables SR-LLM to evolve its expression library and knowledge module adaptively, guided by both data-driven discovery and structured reflection, thereby advancing the frontier of interpretable symbolic regression. For detailed implementation and case studies of Updating-Part, please refer to [SI Appendix, Updating-Part](#).

The radix trees and policy networks are updated in each execution round, while the expression library is updated only after the policy network converges. This delay ensures accurate symbol evaluation, as it relies on a large number of model samples. Moreover, such a delayed update saves much time and reduces computational costs associated with LLM interactions. Besides, the maximum size of the expression library is preselected and fixed. Once the capacity limit of the expression library is reached, symbol scores determine which old symbols are discarded and which new ones are added. New symbols generated by the LLM are returned to the Sampling-Part for further model exploration. The continuous enrichment of the knowledge modules and expression library exemplifies the incremental learning feature of our framework.

Notably, the process above, where RAG enhances generation by integrating existing knowledge and continuously updating the knowledge module and expression library, mirrors the

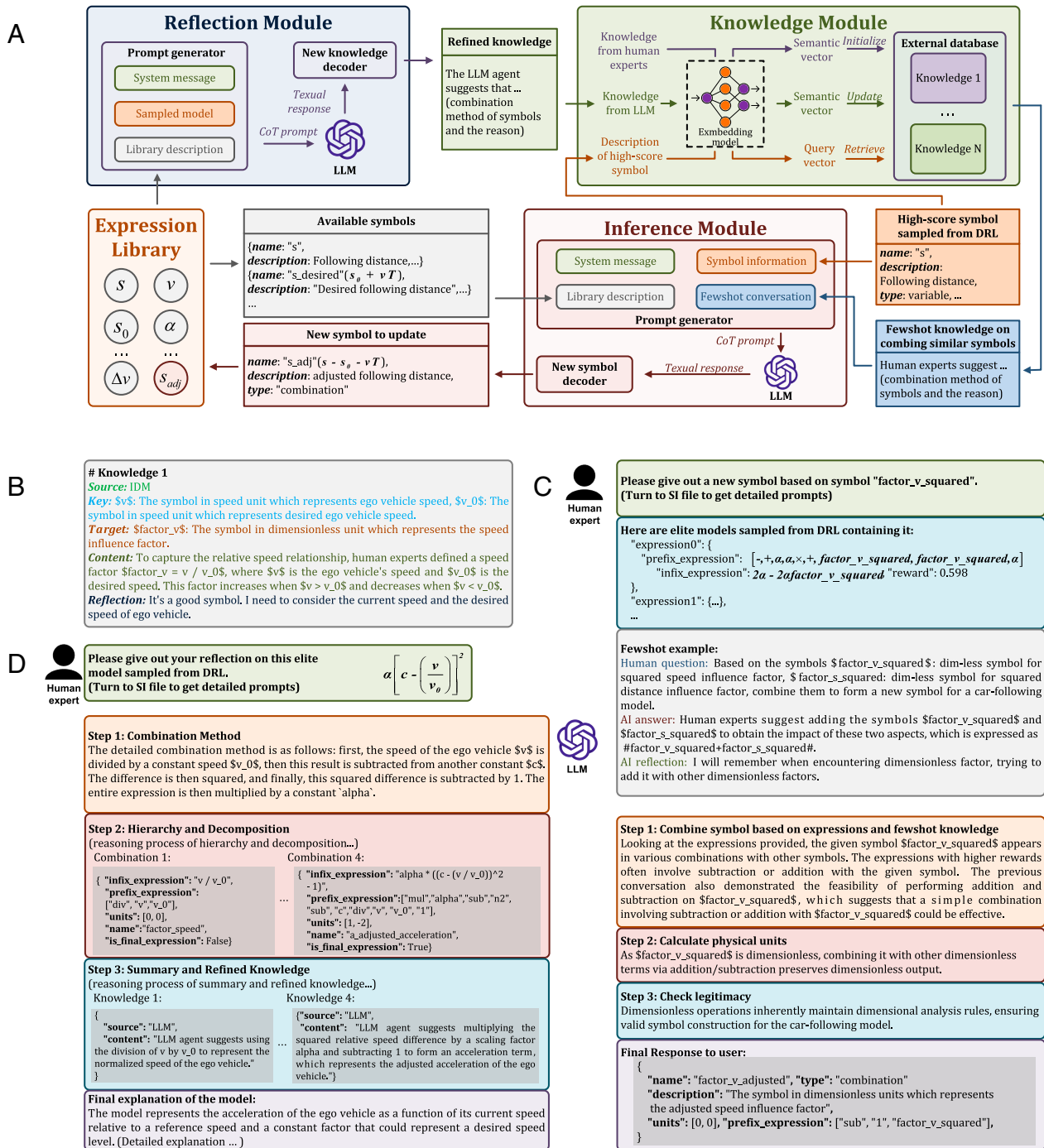


Fig. 3. The detailed implementation of Updating-Part. (A) Training procedure of the knowledge, reasoning, and reflection modules. (B) Detailed examples of Knowledge module. (C) Detailed examples of Inference module (D) Detailed examples of Reflection module.

evolutionary pattern of human society. This pattern involves accumulating and leveraging the experiences of predecessors for production and innovation. This technical approach enables continuous updates through iterative cycles and fully leverages the diversity of knowledge. Additionally, our framework allows for flexible manual specification of the knowledge base content, guiding it toward different evolutionary paths and exploring various excellent models.

SR-LLM possesses three major advantages that help overcome the three obstacles mentioned previously. First, our framework significantly reduces the search space required for symbolic

regression. Introducing new symbols with more complex semantics, composed of combinations of existing symbols, effectively narrows the search space. Let us briefly explain why introducing new symbols with more complex semantics, composed of combinations of existing symbols, can effectively reduce the search space required for symbolic regression. We estimate the search space size using the current expression library's capacity and the minimum length of symbol sequences needed to achieve the target model. Assuming the number of symbols in the current expression library is η , there exists a minimum model length l such that combinations of l symbols from the expression library

can achieve the target model. Therefore, a breadth-first search on a prefix tree with a maximum depth of l guarantees hitting the target model, and the maximum number of models required to be searched is η^l . Based on this, we can further calculate the reduction in the search space when adding new symbols to the expression library. Assuming that after adding η' new symbols to the expression library with a size of η , the shortest length decreases by l' from l , where $\eta' \ll \eta$ and $l - l' \gg 1$. The reduction ratio of the search space can be calculated as $\frac{(\eta+\eta')^{l-l'}}{\eta^l} = \frac{1}{\eta^{l'}}(1 + \frac{\eta'}{\eta})^{l-l'} \approx \frac{1}{\eta^{l'}} \ll 1$. This shows that introducing new symbols, despite increasing the library size, can still significantly reduce the search space if they effectively shorten expressions, making it easier for DRL to hit the target models.

Second, our framework is capable of constructing semantically meaningful and structurally complex analytical expressions. The LLM incrementally adds new symbols to the expression library, while the reflection module interprets and reflects on high-performing expressions discovered by DRL. The outcomes of these reflections are stored in the knowledge module to guide future symbol generation by the LLM subsequently. The interactive process between DRL and LLM enables incremental expansion of both the expression library and knowledge module, ensuring fitting effectiveness and interpretability, which represents one of the most prominent advantages of our SR framework.

Third, our framework is able to reduce the time required for parameter calibration and, when necessary, find the global optimum for parameter calibration. In previous SR methods, the calibration of expressions was typically inflexible and unable to locate the global optimum. For instance, using a fixed parameter calibration strategy such as L-BFGS for all sampled models without considering whether the models had been calibrated before or how accurate the results were, made their parameter calibration processes less efficient especially for noise nonlinear systems. SR-LLM employs a two-stage parameter calibration method and utilizes a prefix tree to store calibration results, allowing direct extraction for existing models. For models not present in the prefix tree, a quick rough calibration is performed first, followed by a decision on whether to conduct a precise calibration based on preliminary fitting error of the model. This significantly improves calibration efficiency of SR-LLM.

Further technical details of the search algorithms, network architecture, and training procedure are described in *Materials and Methods*.

Testing Results on Standard Benchmarks. To evaluate the basic search performance of SR-LLM compared to the baselines, we utilized popular benchmark expressions, including Nguyen (38), Constant (14), Livermore (34), R (39), Vladislavleva (40), Neat (41), Keijzer (42), Korns (43) and Jin (44), collectively referred to as “Fundamental-Benchmark.” This benchmark, devoid of physical dimensions, consists of 100 expressions of varying complexity. We believe these expressions can accurately reflect the fundamental search capacity of each method. Since Fundamental-Benchmark lacks guaranteed physical interpretability, the LLM-based RAG, which relies on physical knowledge, is excluded from this test. We refer to SR-LLM without LLM-based RAG guidance as SR-LLM w/o.

We further evaluate SR-LLM using the “Feynman-Benchmark,” assisted by the LLM. This benchmark comprises 100 classical physics formulas with clear physical meanings, divided into two parts. Ten expressions are selected to provide prior physical knowledge for the LLM-based RAG, while the

remaining 90 are used to test the search effectiveness of various methods. For SR-LLM, we explore 1,000 symbolic models in each epoch. After an average of 10 epochs of DRL training, SR-LLM updates the expression library with specific knowledge, enriches the knowledge module with newly acquired knowledge, and retrain the DRL policy. This iterative process constitutes one evolution, defined as a single cycle of interaction between the DRL and LLM components. We set the maximum number of evolutions to 10, which corresponds to exploring up to 100,000 models in total. To highlight the importance of the LLM, we compare the full SR-LLM with physical knowledge guidance against SR-LLM w/o (without LLM-based knowledge guidance, training for 100 epochs in total) in this benchmark.

We select eight latest symbolic regression methods for comparison with SR-LLM, including PySR (45), GP-GOMEA (46), NGGP (47), gplearn (48), PhySO (5), E2E (49), PSRN (50) and QLattice (51), encompassing five categories: GP, DRL, Pretrain, BF, and DL. For the specific meanings of each category and a more detailed classification, please refer to Table 1.

We use the R^2 score and exact recovery rate to measure symbolic regression performance. Specifically, R^2 score is computed as $R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$, where N denotes the total number of test samples, y_i represents the ground-truth target value for the i -th sample, $\hat{y}_i$ denotes the predicted value produced by the symbolic regression method, and $\bar{y} = \frac{1}{N} \sum_{i=1}^N y_i$ is the mean of the true target values over all N samples. The exact recovery rate is defined as the proportion of test cases in which the discovered symbolic expression is exactly equivalent to the true underlying ground-truth expression, which evaluates the method’s ability to recover the precise symbolic formula.

For each expression in the two benchmarks, we conduct 10 repeated experiments across all methods to minimize randomness from searching process. We plot the mean and the SD of the R^2 score and exact recovery rate from these 10 repetitions in Fig. 4, with the length of error bars representing the SD. For fairness, the 10 Feynman expressions used to provide prior physical knowledge are excluded from the results statistics. Furthermore, in single experiment of an expression, we limit the maximum number of expression explorations to 100,000 for all methods. The only exception is the brute-force searching PSRN algorithm, which explores significantly more expressions in the same considerable time compared to other methods. This approach focuses more on assessing subtree values within a sampling tree rather than evaluating entire expressions, making it unfair to compare its performance based solely on the count of evaluated expressions. To ensure comparability, we allow it to run for the same time duration as PySR’s exploration of 100,000 expressions, mirroring the comparison made in the PSRN (50).

Table 1. Classification of baseline methods

Baseline method	Category	Baseline method	Category
PySR (45)	GP	PhySO (5)	DRL
GP-GOMEA (46)	GP	E2E (49)	Pretrain
NGGP (47)	GP	PSRN (50)	BF
gplearn (48)	GP	QLattice (51)	DL

GP refers to the method of inferring expressions using genetic programming. DRL refers to the approach of acquiring the model’s backbone structure through deep reinforcement learning. Pretrain refers to the method of inferring expressions directly from input data using a pretrained model. BF stands for brute force searching. DL denotes the method of inferring expressions using deep neural networks. Detailed descriptions of the methods are provided in *SI Appendix, Compared Methods*.

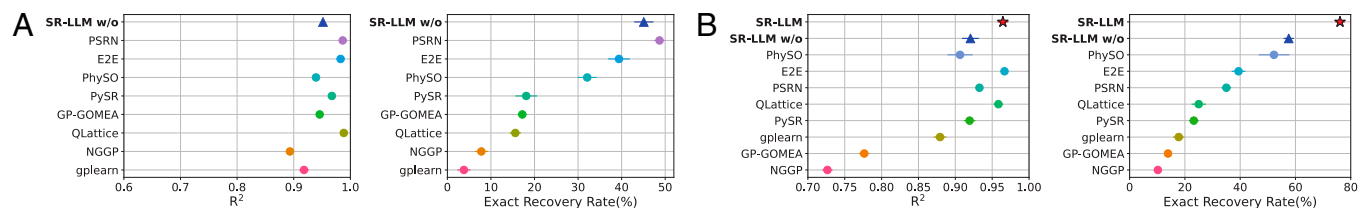


Fig. 4. Test results of SR-LLM and other baseline methods on standard benchmarks. (A) The results on Fundamental-Benchmark. (B) The results on Feynman-Benchmark.

All experiments were conducted on 2 Intel(R) Xeon(R) Platinum 8374B CPUs operating at 2.70 GHz alongside an NVIDIA RTX 4090 GPU, with PSRN leveraging the GPU for acceleration purposes.

As shown in the Fundamental-Benchmark results in Fig. 4A, SR-LLM w/o surpasses all previous symbolic regression schemes except PSRN in exact recovery rate under the same number of explored expressions, indicating that SR-LLM w/o has achieved the best fundamental search capacity among symbolic regression methods. It is noteworthy that SR-LLM lags behind brute-force searching-based PSRN in terms of both R^2 score and exact recovery rate. This discrepancy primarily stems from PSRN's efficient exploration capability, which allows it to evaluate a significantly larger number of models. However, the DRL search method, which can easily incorporate beneficial priors at each expression construction step to guide the search, is better suited for the LLM-integrated, knowledge-driven search paradigm. Additionally, SR-LLM provides a comprehensive library of fundamental operators, improving search performance. In summary, SR-LLM achieves the highest standard in fundamental search capability within the DRL paradigm, thereby providing a strong foundation for subsequent incorporation of knowledge-driven retrieval-augmented generation.

As demonstrated by the Feynman-Benchmark results in Fig. 4B, SR-LLM w/o, without knowledge-driven assistance, has already surpassed other baselines in exact recovery rate for interpretable, physically meaningful analytical models. With LLM-based knowledge-driven assistance, SR-LLM achieves a 76.1% recovery rate on this benchmark, significantly outperforming other symbolic regression methods. This result demonstrates the capability of our knowledge-driven symbolic regression framework that integrates LLMs with RAG in enhancing symbolic regression performance. Interestingly, PhySO, which exhibits moderate performance on the Fundamental-Benchmark in the absence of physical unit constraints, achieves superior results on the Feynman-Benchmark when unit constraints are imposed, surpassing both PSRN and EZE in exact recovery rate. This observation indicates that incorporating physical unit constraints helps guide the search toward dimensionally consistent expressions, effectively reducing the search space and filtering out implausible candidates. This highlights the importance and effectiveness of integrating unit constraints into our fundamental search framework.

The Discovered Models and Rediscovered Classic Car-Following Models by SR-LLM. To further evaluate the ability of SR-LLM to accelerate search by integrating existing knowledge with real-world empirical data, we apply it to build car-following models using the open-access trajectory dataset from the NGSIM program. We select 108 car-following vehicle pairs to demonstrate the capability of SR-LLM, further dataset details are provided in [SI Appendix, NGSIM Dataset Introduction](#). Due to the

measurement noise (20, 21) and variability in driving styles and driving scenarios (22, 23), these trajectories are laden with significant biased noise, which poses substantial challenges for performing symbolic regression on such real-world datasets, whether it be identifying new, meaningful models or rediscovering classic car-following models.

Unlike standard benchmarks, the ground-truth formula for the car-following problem is unknown. We rely on real-world empirical data to construct explicitly meaningful analytic models with excellent fitting performance. In this domain, human experts have proposed several well-known classic car-following models, among which we selected the representative Helly (52, 53), GHR (54), and IDM (55) models. These models are renowned and widely used in the field of car-following, not only for their good representation of following behavior but also for their clear physical meanings. Explanations of these models are provided in [SI Appendix, Eqs. 1–3](#).

Our main goal is to demonstrate that SR-LLM can integrate knowledge from three classical expert models to discover novel and high-performing car-following models that have not been previously proposed in the literature. At the same time, as a byproduct of the discovery process, we show that when prior knowledge from expert models is provided, SR-LLM can achieve human-expert-level insight, recovering the underlying expert models accurately even from real trajectory data with high levels of biased noise. We believe this capability positions SR-LLM as a powerful tool for scientific discovery across a broad range of disciplines, offering valuable insights and inspiration for researchers in diverse fields.

In the experimental setup, to ensure thorough integration of knowledge from the three expert models, we not only incorporate their combined knowledge into the SR-LLM knowledge module but also consider the similarity score to each of the expert models into the model evaluation metric.

For the baseline comparison, we select PhySO (5) due to its explicit incorporation of physical dimensionality constraints, which simultaneously ensures dimensional consistency and interpretability for complex car-following models. Crucially, PhySO also demonstrates strong performance on standard benchmarks, making it an ideal reference point for evaluation of our approach. Notably, to incorporate expert prior knowledge into PhySO and ensure a fairer comparison, we also augment PhySO's reward function with the same complexity and similarity metric to the expert models used in SR-LLM. The initial symbol library (derived from the symbols contained in expert models), the total number of exploration trials, and other configurations are also kept consistent. Similar to the experiments on Feynman-Benchmark, we employ an incremental learning strategy with a total of 50 evolutions, each consisting of 20 epochs, and sample 1,000 models per epoch, resulting in a maximum exploration budget of one million models.

The incremental learning framework provides significant benefits for enhancing the performance of SR-LLM. Fig. 5A

Table 2. The highest-scoring models discovered by PhySO and SR-LLM from the first 500,000 explored expressions

Model name	Model expression from PhySO	Fitting score		Model expression from SR-LLM	Fitting score	
		Position ↑	Speed ↑		Position ↑	Speed ↑
Discovered Model 1	$a_n(t) = \alpha - \frac{\Delta v}{s}$	0.706	0.356	$a_n(t) = \alpha[(c - \frac{v}{v_0})^2 - 1]$	0.829	0.511
Discovered Model 2	$a_n(t) = \frac{\Delta v s(a + k_1 v)}{v(\Delta v T - s)}$	0.664	0.338	$a_n(t) = b + \frac{b}{2c - \frac{v\Delta v}{2\sqrt{ab}} + 1} - \alpha\sqrt{\frac{v}{v_0}}$	0.826	0.498
Discovered Model 3	$a_n(t) = \frac{\Delta v(\Delta v + v_0)}{s}$	0.630	0.323	$a_n(t) = \alpha[c(c - \frac{v}{v_0})^2 - (\frac{v}{v_0}) - 1] + \frac{v\Delta v}{s}$	0.802	0.448

Red constants in the table represent parameters to calibrate, while the rest are input variables.

shows the training results for each evolution in incremental learning, where each evolution consists of 20 epochs and ends with an update to the expression library. The figure displays the average reward and the reward distribution of the top 10% models after every 20 epochs. The reward distribution of candidate models is estimated using a Gaussian kernel (55). It is evident that the introduction of new symbols improves model performance, evidenced by both higher average rewards for the top 10% models and a shift in the model distribution toward higher rewards. This demonstrates our approach is capable of more rapidly identifying models with superior performance, with training stability improving as new symbols are added.

As the primary objective of our experiments, SR-LLM discovers the car-following models which have never been mentioned in the last 70 y of research. In Table 2, we present three car-following models with the highest scores, identified by SR-LLM and PhySO within the first 500,000 trials, the detailed information of used symbols is provided in *SI Appendix, Eqs. 1–3*. Their average fitting scores are evaluated on 108 car-following trajectories from the NGSIM dataset, with the methodology detailed in *SI Appendix, Eq. 12*. Compared with PhySO, the models discovered by SR-LLM better integrate multiple factors from driving scenarios, achieving significantly higher fitting accuracy. Moreover, SR-LLM ensures clear compositional structures and meaningful interpretations, as exemplified by Discovered Model 1 in Fig. 3*D*. In addition, SR-LLM can absorb expert knowledge and innovate beyond it. For instance, compositional terms in Discovered Model 2, such as $\frac{v}{v_0}$ and $\frac{v\Delta v}{2\sqrt{ab}}$, are derived from IDM knowledge but are combined in novel ways.

Three discovered models obtained by SR-LLM also outperform the Helly GHR, and IDM in terms of fitting score, as listed in Table 3. Interestingly, the new models overcome certain limitations of the classic expert models. Fig. 5*B* illustrates the observed and simulated car-following dynamics of a selected

vehicle pair (preceding vehicle ID: 255, ego vehicle ID: 257) in the US 101 dataset between 7:50 AM and 8:05 AM. While both the IDM and Discovered Models found by SR-LLM track the trajectory of following vehicles in the early stages, the IDM model fails during sudden relative speed changes, leading to significant errors in relative spacing between two consecutive vehicles. This may result from overdeceleration feature of IDM, which is one of the well-known limitations of IDM (56, 57). This highlights the potential of SR-LLM in enhancing fitting performance when applied to real-world, challenging data scenarios, while ensuring interpretability at the same time.

As illustrated in Table 3, during the process of discovering novel models, SR-LLM demonstrates human-expert-level insight with prior knowledge, accurately recovering the underlying expert models even from real trajectory data with high levels of biased noise. Although PhySO employs the same similarity metric, it fails to rediscover the Helly and IDM within a maximum of one million trials, while SR-LLM successfully rediscovers them. These results highlight the superiority of SR-LLM in understanding and rediscovering prior research, as well as its capability to overcome significant real-world biased noise, thereby deriving concise and interpretable dynamic physical models.

Discussion

In this work, we propose SR-LLM, a symbolic regression framework to integrate retrieval-augmented generation with large language models for incremental learning. SR-LLM encodes human prior knowledge and historical discoveries as structured knowledge in an external database, which is retrieved and applied to guide the search process. Guided by LLMs, the system decomposes knowledge into symbolic primitives and uses deep reinforcement learning to assemble them into complex, interpretable analytic expressions. We further enhance performance

Table 3. The rediscovery of famous traditional car-following models occurs alongside the discovery of novel models

Model source	Model expression	Run time (min)		Trial expressions		Fitting score	
		PhySO	SR-LLM	PhySO	SR-LLM	Position ↑	Speed ↑
Helly (1959)	$a_n(t) = k_1 \Delta v_{n-1,n}(t - \tau) + k_2 [\Delta x_{n-1,n}(t - \tau) - D_n(t)]$ where $\Delta v_{n-1,n}(t - \tau) = v_{n-1}(t - \tau) - v_n(t - \tau)$ $\Delta x_{n-1,n}(t - \tau) = x_{n-1}(t - \tau) - x_n(t - \tau)$ $D_n(t) = 20 + \beta v_n(t - \tau)$	Fail	6.85	Fail	43,000	0.315	0.158
GHR (1961)	$a_n(t) = c v_n^m(t) \frac{\Delta v_{n-1,n}(t - \tau)}{\Delta x_{n-1,n}(t - \tau)}$	0.43	0.18	14,000	5,000	0.669	0.347
IDM (2000)	$a_n(t) = \alpha[1 - (\frac{v}{v_0})^\delta - (\frac{s^*(v, \Delta v)}{s})^2]$ where $s^*(v, \Delta v) = s_0 + vT + \frac{v\Delta v}{2\sqrt{ab}}$ $v = v_n(t), \Delta v = v_n(t) - v_{n-1}(t)$ $s = x_{n-1}(t) - x_n(t) - l_n$	Fail	133.3	Fail	502,000	0.660	0.274

We set the maximum number of explored models to one million. Both the Helly and IDM models are represented as single, unified formulas. Red constants in the table represent parameters to calibrate, while the rest are input variables. For simplicity, and following prior references (54, 55), we fix the blue constants as $m = 1, l = 1, \delta = 4$.

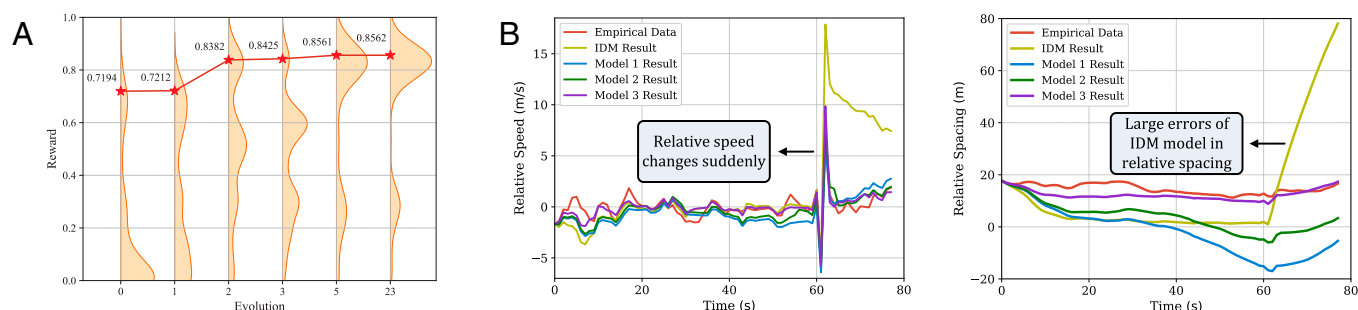


Fig. 5. Experimental results of symbolic regression on car-following data. (A) The average and distribution of rewards for the top 10% models. (B) A comparison of vehicle speed and position time series replicated by different models. The *Left* panel shows the relative speed replicated by the Intelligent Driver Model (IDM) and the models discovered by SR-LLM and listed in Table 2, while the *Right* panel displays the corresponding relative spacing reproduced by the same set of models.

through tailored improvements to the DRL algorithm, parameter calibration, and reward design. Extensive experiments show that SR-LLM achieves state-of-the-art results on benchmark datasets and discovers novel, semantically meaningful car-following models with superior fit, demonstrating its effectiveness as a knowledge-driven, LLM-based symbolic regression framework.

The greatest strength of SR-LLM lies in its ability to effectively integrate human prior knowledge with past exploration results, enabling symbolic regression to build upon the shoulders of giants. This can be analogized to DeepMind's AlphaGo (17) introduced in 2016: just as AlphaGo leveraged human's game records to guide its exploration of meaningful strategies, SR-LLM utilizes expert knowledge to inform its search for meaningful model structures. This is empirically validated in our car-following modeling task. Given prior knowledge of three classical car-following models, SR-LLM fully absorbs the compositional principles embedded in these expert models during the search process. It incrementally constructs increasingly complex symbolic combinations grounded in expert knowledge, ultimately discovering novel car-following models with clear compositional semantics and excellent fitting performance.

Indeed, we also note several emerging works that apply large language models to symbolic regression. Early approaches modeled SR as a language task, using Transformer-based sequence generation to directly predict expressions (49, 58, 59). However, these methods fail to leverage the commonsense knowledge and reasoning capabilities of LLMs, relying heavily on extensive training data. Subsequently, with the rise of large language models, fine-tuning LLMs (14) or relying solely on instruction-based interaction (12, 60, 61) for SR has emerged. Yet, these approaches are constrained by the LLM's static prior knowledge boundaries and, lacking guided integration of external knowledge, often produce models that are difficult for humans to interpret. Although some conceptual knowledge-guided SR methods have been proposed (16), the knowledge they employ lacks formal verification of abstract concepts and may not be effective when the required knowledge is highly complex.

The core innovation of SR-LLM lies in proposing a hybrid symbolic regression framework combining RAG and DRL. By dynamically retrieving historical exploration results from an external knowledge base, it constructs symbolic components and assembles them into complex expressions. Our core differentiating contributions lies in three key differentiators: 1) Unlike prior methods that rely solely on general-purpose symbolic reasoning within the LLM, SR-LLM grounds its inference in domain-specific symbolic combinations retrieved from a curated knowledge base. Since the LLM's generic priors

may not align with the structural or semantic constraints of specialized domains, such as car-following dynamics, this retrieval-based grounding ensures relevance and interpretability. 2) Rather than relying on expensive reward based fine-tuning to symbolic representations, SR-LLM leverages RAG to directly incorporate symbolic composition rules as actionable knowledge. This enables incremental, reusable knowledge transfer across tasks, significantly improving knowledge utilization efficiency and ensuring steady, interpretable progress in model accuracy. 3) Models that generated by SR-LLM are easily interpretable in terms of their meaning, not only because the intermediate steps in the growth process involve semantically meaningful symbolic combinations but also because high-performing formulas discovered by SR-LLM are hierarchically explained with the help of the LLM. The fusion of prior knowledge with dynamic exploration to learn incrementally, rather than static reliance on the LLM, is the core innovation that distinguishes this work from other SR+LLM approaches.

While it is true that SR-LLM currently leverages pretrained LLM knowledge as prior guidance for efficient symbolic space navigation, we emphasize that the framework's architecture inherently supports cold-start operation without human-provided priors. Similar to AlphaZero's paradigm (62), which bootstrapped superhuman performance through iterative self-refinement starting from purely random moves, our framework could be adapted to autonomously generate its own initial knowledge base. This would involve: 1) conducting stochastic exploration of elementary symbolic combinations in a controlled complexity space, and 2) using reinforcement signals from validation metrics to iteratively distill promising candidate expressions into emergent priors. These self-discovered priors could then seamlessly integrate into our current framework's reasoning pipeline, enabling progressive knowledge crystallization without human intervention. While this self-bootstrapping capability requires further development, the present implementation already provides a critical bridge between knowledge-guided search and eventual autonomous discovery. By allowing scientists to flexibly combine domain-specific priors with exploratory search, SR-LLM serves as both a practical tool for immediate scientific advancement and a foundational architecture for future fully autonomous symbolic AI systems.

Materials and Methods

More Details of SR-LLM. The detailed implementation of SR-LLM, additional information, and results, including its robustness under Gaussian noise (1 and 10%) on standard benchmarks and its performance on a fully random

dataset to exclude the influence of LLM pretraining priors, are provided in [SI Appendix](#).

Data, Materials, and Software Availability. Open-source experimental data and codes are available at <https://github.com/ThuOneLab/SR-LLM> (63).

ACKNOWLEDGMENTS. This work was supported in part by the National Key Research and Development Program of China (Grant No. 2023YFB2504400), the Science and Technology Development Fund, Macao Special Administrative Region under Grants 0093/2023/R1A2, 0145/2023/R1A3 and 0157/2024/R1A2, and the QAIL Grant for Decentralized Science Center of Parallel Intelligence (#QAIL-2024-0906) at Obuda University

1. R. Iten, T. Metger, H. Wilming, L. Del Rio, R. Renner, Discovering physical concepts with neural networks. *Phys. Rev. Lett.* **124**, 010508 (2020).
2. M. Cranmer *et al.*, Discovering symbolic models from deep learning with inductive biases. *Adv. Neural Inf. Process. Syst.* **33**, 17429–17442 (2020).
3. M. Schmidt, H. Lipson, Distilling free-form natural laws from experimental data. *Science* **324**, 81–85 (2009).
4. S. M. Udrescu, M. Tegmark, Symbolic pregression: Discovering physical laws from distorted video. *Phys. Rev. E* **103**, 043307 (2021).
5. W. Tenachi, R. Ibata, F. I. Diakogiannis, Deep symbolic regression for physics guided by units constraints: Toward the automated discovery of physical laws. *Astrophys. J.* **959**, 99 (2023).
6. P. A. Kamienny, G. Lample, S. Lamprier, M. Virgolin, "Deep generative symbolic regression with Monte-Carlo-tree-search" in *International Conference on Machine Learning* (PMLR, 2023), pp. 15655–15668.
7. P. Lewis *et al.*, Retrieval-augmented generation for knowledge-intensive NLP tasks. *Adv. Neural Inf. Process. Syst.* **33**, 9459–9474 (2020).
8. J. Achiam *et al.*, GPT-4 technical report. arXiv [Preprint] (2023). <https://arxiv.org/abs/2303.08774> (Accessed 8 December 2025).
9. R. Anil *et al.*, Palm 2 technical report. arXiv [Preprint] (2023). <https://arxiv.org/abs/2305.10403> (Accessed 8 December 2025).
10. T. Brown *et al.*, Language models are few-shot learners. *Adv. Neural Inf. Process. Syst.* **33**, 1877–1901 (2020).
11. C. Chang *et al.*, Driving-rag: Driving scenarios embedding, search, and rag applications. arXiv [Preprint] (2025). <https://arxiv.org/abs/2504.04419> (Accessed 8 December 2025).
12. M. Merler, K. Haitsiukevich, N. Dainese, P. Martinen, In-context symbolic regression: Leveraging large language models for function discovery. arXiv [Preprint] (2024). <https://arxiv.org/abs/2404.19094> (Accessed 8 December 2025).
13. S. Sharlin, T. R. Josephson, In context learning and reasoning for symbolic regression with large language models. arXiv [Preprint] (2024). <https://arxiv.org/abs/2410.17448> (Accessed 8 December 2025).
14. Y. Li *et al.*, MLLM-SR: Conversational symbolic regression base multi-modal large language models. arXiv [Preprint] (2024). <https://arxiv.org/abs/2406.05410> (Accessed 8 December 2025).
15. P. Shojaaee, K. Meidani, S. Gupta, A. B. Farimani, C. K. Reddy, LLM-SR: Scientific equation discovery via programming with large language models. arXiv [Preprint] (2024). <https://arxiv.org/abs/2404.18400> (Accessed 8 December 2025).
16. A. Grayeli, A. Sehgal, O. Costilla Reyes, M. Cranmer, S. Chaudhuri, Symbolic regression with a learned concept library. *Adv. Neural Inf. Process. Syst.* **37**, 44678–44709 (2024).
17. D. Silver *et al.*, Mastering the game of go with deep neural networks and tree search. *Nature* **529**, 484–489 (2016).
18. H. Zhao *et al.*, Explainability for large language models: A survey. *ACM Trans. Intell. Syst. Technol.* **15**, 1–38 (2024).
19. C. Singh, J. P. Inala, M. Galley, R. Caruana, J. Gao, Rethinking interpretability in the era of large language models. arXiv [Preprint] (2024). <https://arxiv.org/abs/2402.01761> (Accessed 8 December 2025).
20. B. Coifman, L. Li, A critical evaluation of the next generation simulation (NGSIM) vehicle trajectory dataset. *Transp. Res. Part B: Methodol.* **105**, 363–377 (2017).
21. C. Thiemann, M. Treiber, A. Kesting, Estimating acceleration and lane-changing dynamics from next generation simulation trajectory data. *Transp. Res. Rec.* **2088**, 90–101 (2008).
22. S. An, L. Xu, G. Chen, Z. Shi, A new car-following model on complex road considering driver's characteristics. *Mod. Phys. Lett. B* **34**, 2050182 (2020).
23. J. W. Ro, P. S. Roop, A. Malik, P. Ranjitkar, A formal approach for modeling and simulation of human car-following behavior. *IEEE Trans. Intell. Transp. Syst.* **19**, 639–648 (2017).
24. J. Weizenbaum, Eliza—A computer program for the study of natural language communication between man and machine. *Commun. ACM* **9**, 36–45 (1966).
25. A. Newell, H. Simon, The logic theory machine—A complex information processing system. *IRE Trans. Inf. Theory* **2**, 61–79 (1956).
26. T. Winograd, Procedures as a representation for data in a computer program for understanding natural language (Tech. Rep., 1971).
27. R. Evans, E. Grefenstette, Learning explanatory rules from noisy data. *J. Artif. Intell. Res.* **61**, 1–64 (2018).
28. D. R. Morrison, Patricia-practical algorithm to retrieve information coded in alphanumeric. *J. ACM* **15**, 514–534 (1968).
29. C. B. Browne *et al.*, A survey of Monte Carlo tree search methods. *IEEE Trans. Comput. Intell. AI Games* **4**, 1–43 (2012).
30. S. Hochreiter, J. Schmidhuber, Long short-term memory. *Neural Comput.* **9**, 1735–1780 (1997).
31. C. Zhu, R. H. Byrd, P. Lu, J. Nocedal, Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization. *ACM Trans. Math. Software (TOMS)* **23**, 550–560 (1997).
32. D. C. Liu, J. Nocedal, On the limited memory BFGS method for large scale optimization. *Math. Prog.* **45**, 503–528 (1989).

Author affiliations: ^aDepartment of Automation, Tsinghua University, Beijing 100084, China; ^bState Key Laboratory of Multimodal Artificial Intelligence Systems, Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China; ^cSCAN Center, University of Glasgow, Glasgow, G12 8QB, United Kingdom; ^dDepartment of Engineering, University of Cambridge, Cambridge, CB2 1PZ, United Kingdom; ^eJohn von Neumann Faculty of Informatics, Obuda University, Budapest H-1034, Hungary; ^fDepartment of Electrical Engineering Systems, University of Southern California, Los Angeles, CA 90007; ^gThe DeSci Center of Parallel Intelligence, Obuda University, Budapest H-1034, Hungary; ^hMacau Institute of Systems Engineering, Macau University of Science and Technology, Macao 999078, China; and ⁱState Key Laboratory for Management and Control of Complex Systems, Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China

Author contributions: L.L. designed research; Z.G., and S.W. performed research; Z.G., S.W., and L.L. analyzed data; and Z.G. S.W., Y.T., J.Y., H.Y., X.N., L.K., L.L., P.A.I., and F.-Y.W. wrote the paper.

33. L. Li, X. M. Chen, L. Zhang, A global optimization algorithm for trajectory data based car-following model calibration. *Transp. Res. Part C: Emerg. Technol.* **68**, 311–332 (2016).
34. B. K. Petersen *et al.*, Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients. arXiv [Preprint] (2019). <https://arxiv.org/abs/1912.04871> (Accessed 8 December 2025).
35. R. S. Sutton, D. McAllester, S. Singh, Y. Mansour, Policy gradient methods for reinforcement learning with function approximation. *Adv. Neural Inf. Process. Syst.* **12**, 1057–1063 (1999).
36. T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor" in *International Conference on Machine Learning* (PMLR, 2018), pp. 1861–1870.
37. T. Schaul, J. Quan, I. Antonoglou, D. Silver, Prioritized experience replay. arXiv [Preprint] (2015). <https://arxiv.org/abs/1511.05952> (Accessed 8 December 2025).
38. N. Q. Uy, N. X. Hoai, M. O'Neill, R. I. McKay, E. Galván-López, Semantically-based crossover in genetic programming: Application to real-valued symbolic regression. *Genet. Program. Evolvable Mach.* **12**, 91–119 (2011).
39. K. Krawiec, T. Pawlak, "Approximating geometric crossover by semantic backpropagation" in *Proceedings of the 15th Annual Conference on Genetic and Evolutionary Computation* (2012), pp. 941–948.
40. E. J. Vladislavleva, G. F. Smits, D. Den Hertog, Order of nonlinearity as a complexity measure for models generated by symbolic regression via pareto genetic programming. *IEEE Trans. Evol. Comput.* **13**, 333–349 (2008).
41. L. Trujillo, L. Muñoz, E. Galván-López, S. Silva, neat genetic programming: Controlling bloat naturally. *Inf. Sci.* **333**, 21–43 (2016).
42. M. Keijzer, "Improving symbolic regression with interval arithmetic and linear scaling" in *European Conference on Genetic Programming*, C. Ryan, T. Soule, M. Keijzer, E. Tsang, R. Poli, E. Costa, Eds. (Springer, 2003), pp. 70–82.
43. M. F. Korns, "Accuracy in symbolic regression" in *Genetic Programming Theory and Practice IX*, R. Riolo, E. Vladislavleva, J. H. Moore, Eds. (Springer, 2011), pp. 129–151.
44. Y. Jin, W. Fu, J. Kang, J. Guo, J. Guo, Bayesian symbolic regression. arXiv [Preprint] (2019). <https://arxiv.org/abs/1910.08892> (Accessed 8 December 2025).
45. M. Cranmer, Interpretable machine learning for science with PYSR and symbolicregression.jl. arXiv [Preprint] (2023). <https://arxiv.org/abs/2305.01582> (Accessed 8 December 2025).
46. M. Virgolin, T. Alderliesten, C. Witteveen, P. A. Bosman, Improving model-based genetic programming for symbolic regression of small expressions. *Evol. Comput.* **29**, 211–237 (2021).
47. T. Mundhenk *et al.*, Symbolic regression via deep reinforcement learning enhanced genetic programming seeding. *Adv. Neural Inf. Process. Syst.* **34**, 24912–24923 (2021).
48. T. Stephens, gplearn: Genetic programming in python with a scikit-learn inspired api. GitHub. <https://github.com/trevorstephens/gplearn>. (Accessed 8 December 2025).
49. P. A. Kamienny, S. d'Ascoli, G. Lample, F. Charton, End-to-end symbolic regression with transformers. *Adv. Neural Inf. Process. Syst.* **35**, 10269–10281 (2022).
50. K. Ruan *et al.*, Discovering symbolic expressions with parallelized tree search. arXiv [Preprint] (2024). <https://doi.org/10.48550/arXiv.2407.04405> (Accessed 8 December 2025).
51. K. R. Broløs *et al.*, An approach to symbolic regression using FEYN. arXiv [Preprint] (2021). <https://arxiv.org/abs/2104.05417> (Accessed 8 December 2025).
52. W. Helly, Simulation of bottlenecks in single-lane traffic flow. *Theory Traffic Flow*, 207–238 (1959).
53. W. La Cava, K. Danai, L. Spector, Inference of compact nonlinear dynamic models by epigenetic local search. *Eng. Appl. Artif. Intell.* **55**, 292–306 (2016).
54. D. Gaizs, Non linear follow the leader model of traffic flow. *J. Oper. Res.* **9**, 545–567 (1961).
55. M. Treiber, A. Hennecke, D. Helbing, Congested traffic states in empirical observations and microscopic simulations. *Phys. Rev. E* **62**, 1805 (2000).
56. O. Derbel, T. Peter, H. Zebiri, B. Mourllion, M. Basset, Modified intelligent driver model for driver safety and traffic stability improvement. *IFAC Proc.* **46**, 744–749 (2013).
57. S. Albeaik *et al.*, Limitations and improvements of the intelligent driver model (IDM). *SIAM J. Appl. Dyn. Syst.* **21**, 1862–1892 (2022).
58. M. Valipour, B. You, M. Panju, A. Ghodsi, SymbolicGPT: A generative transformer model for symbolic regression. arXiv [Preprint] (2021). <https://arxiv.org/abs/2106.14131> (Accessed 8 December 2025).
59. P. Shojaaee, K. Meidani, A. Barati Farimani, C. Reddy, Transformer-based planning for symbolic regression. *Adv. Neural Inf. Process. Syst.* **36**, 45907–45919 (2023).
60. Y. Zhu, Z. Y. Khoo, J. S. C. Low, S. Bressan, A personalised learning tool for physics undergraduate students built on a large language model for symbolic regression (IEEE, 2024), pp. 38–43.
61. H. Zhang, Q. Chen, B. Xue, M. Zhang, LLM-META-SR: Learning to evolve selection operators for symbolic regression. arXiv [Preprint] (2025). <https://arxiv.org/abs/2505.18602> (Accessed 8 December 2025).
62. D. Silver *et al.*, Mastering the game of go without human knowledge. *Nature* **550**, 354–359 (2017).
63. Z. Guo *et al.*, Official implementation of SR-LLM: An incremental symbolic regression framework driven by LLM-based retrieval-augmented generation. GitHub. <https://github.com/ThuOneLab/SR-LLM>. (Accessed 8 December 2025).